

Prüfungsleistung: Entwicklung einer Datenbankanwendung

Lehrveranstaltung	Entwicklung einer Datenbankanwendung
Prüfungsform	Gruppenprojekt mit Bericht und Präsentation
Gruppengröße	1 bis 3 Personen
Abgabe	Letzter Vorlesungstag
Präsentation	Letzter Vorlesungstag
Bearbeitungszeitraum	Erster bis letzter Vorlesungstag

Auftrag

Entwickelt eine kleine Datenbankanwendung für einen konkreten Fachfall. Das Ergebnis ist eine ausführbare REST-API mit relationaler Datenbank. Ihr modelliert den Fachfall, begründet die wichtigsten Entscheidungen und weist die Funktion der Anwendung mit automatisierten Tests und einer kurzen Live-Demo nach.

Ihr arbeitet allein oder in einer Gruppe mit höchstens drei Personen. Eine Gruppe gibt eine gemeinsame Implementierung und einen gemeinsamen Bericht ab. In der Präsentation beteiligt sich jedes Gruppenmitglied und beantwortet Fragen zur gesamten Lösung.

Themenwahl

Wählt eine der folgenden vereinfachten Anwendungen. Die Namen bezeichnen nur den Fachfall. Ihr sollt weder die vollständige Plattform nachbauen noch deren aktuelle interne Technik erraten. Ein eigenes Thema ist nach vorheriger Absprache möglich.

Grenzt den Fachfall so ab, dass ihr ihn im Bearbeitungszeitraum sauber umsetzen könnt. Ihr müsst nicht jeden genannten Prozess implementieren. Die beschriebenen Daten und Regeln sind der Ausgangspunkt für euer Modell.

1. TikTok

Modelliert eine Plattform für kurze Videos. Konten veröffentlichen Videos, folgen anderen Konten und reagieren auf Videos mit Kommentaren oder positiven Bewertungen. Ein Video kann mehrere Hashtags tragen. Nutzernamen und Video-IDs sind eindeutig. Ein Konto darf einem anderen Konto höchstens einmal folgen und ein Video höchstens einmal bewerten. Gesperrte Videos bleiben gespeichert, erscheinen aber nicht in der öffentlichen Liste.

Geeignete Hauptressource: `Video`.

2. X, vormals Twitter

Modelliert einen Kurznachrichtendienst. Konten veröffentlichen Beiträge mit einer von euch festgelegten Maximallänge. Beiträge können Antworten auf andere Beiträge sein, erneut geteilt und positiv bewertet werden. Konten können einander folgen. Eine Antwortkette darf beliebig tief sein, muss aber auf einen vorhandenen Beitrag verweisen. Pro Konto und Beitrag ist höchstens eine positive Bewertung erlaubt. Gelöschte Beiträge können als Platzhalter erhalten bleiben, damit Antwortketten lesbar bleiben.

Geeignete Hauptressource: `Post`.

3. Instagram

Modelliert eine Plattform für Bild- und Videobeiträge. Konten veröffentlichen Beiträge, kommentieren und bewerten sie und folgen anderen Konten. Direkte Nachrichten gehören zu einer Unterhaltung zwischen mindestens zwei Konten. Ein Beitrag kann mehrere Mediendateien in einer festen Reihenfolge enthalten. Ein Konto darf denselben Beitrag höchstens einmal bewerten. Private Konten erfordern eine bestätigte Folgeanfrage, bevor ihre Beiträge sichtbar werden.

Geeignete Hauptressource: `MediaPost`.

4. Pinterest

Modelliert eine Anwendung für digitale Pinnwände. Konten erstellen Pinnwände und speichern dort Pins mit Bild, Zieladresse und Beschreibung. Derselbe Pin kann auf mehreren Pinnwänden liegen. Die Zuordnung speichert Zeitpunkt und Position. Konten können anderen Konten oder einzelnen Pinnwänden folgen. Eine private Pinnwand ist nur für ihren Eigentümer und ausdrücklich eingeladene Konten sichtbar.

Geeignete Hauptressource: `Pin` oder `Board`.

5. Tinder

Modelliert eine Dating-Anwendung. Konten pflegen ein Profil mit Bildern und Suchpräferenzen. Eine Wischentscheidung verbindet zwei Konten und enthält Richtung und Zeitpunkt. Erst zwei gegenseitige positive Entscheidungen erzeugen ein Match. Nachrichten dürfen nur zu einem aktiven Match gespeichert werden. Ein Konto darf dieselbe Person nicht mehrfach bewerten, solange die frühere Entscheidung aktiv ist. Blockierte Konten dürfen kein neues Match bilden.

Geeignete Hauptressource: `Match`.

6. Reddit

Modelliert eine Diskussionsplattform mit thematischen Communities. Konten treten Communities bei und veröffentlichen dort Text- oder Bildbeiträge. Kommentare können auf einen Beitrag oder einen anderen Kommentar antworten. Konten bewerten Beiträge und Kommentare positiv oder negativ, jedoch jeweils höchstens einmal. Rollen wie Mitglied und Moderation gehören zur Mitgliedschaft. Entfernte Inhalte bleiben für Moderationszwecke gespeichert, werden aber öffentlich nicht vollständig ausgegeben.

Geeignete Hauptressource: `CommunityPost`.

7. Amazon

Modelliert einen Marktplatz. Händler bieten Produkte zu Preis und verfügbarer Menge an. Dasselbe Produkt kann Angebote mehrerer Händler haben. Eine Bestellung gehört zu einem Kunden und enthält Positionen mit Menge sowie dem Preis zum Kaufzeitpunkt. Dieser Preis darf sich nicht ändern, wenn ein Händler später sein Angebot ändert. Beim Bestellen werden Bestand und Bestellung gemeinsam geändert. Eine Bestellung durchläuft festgelegte Zustände und kann mehrere Sendungen haben.

Geeignete Hauptressource: `Order`.

8. Deutsche Bahn

Modelliert eine Anwendung für Zugverbindungen und Tickets. Bahnhöfe werden durch Zugläufe in einer geordneten Folge verbunden. Jeder Halt hat geplante Ankunft und Abfahrt. Eine Verbindung kann aus mehreren Teilstrecken bestehen. Ein Ticket gehört zu einer Buchung, einer reisenden Person und einer konkreten Verbindung. Abfahrtsbahnhof und Zielbahnhof müssen in dieser Reihenfolge im Zuglauf liegen. Eine Sitzplatzreservierung darf für denselben Zugabschnitt nicht doppelt vergeben werden.

Geeignete Hauptressource: `Journey` oder `Ticket`.

9. Chess.com

Modelliert eine Plattform für Schachpartien. Konten spielen Partien mit Farbe, Zeitkontrolle und Ergebnis. Züge werden in ihrer Reihenfolge mit Notation und Zeitverbrauch gespeichert. Nach einer gewerteten Partie ändern sich beide Wertungszahlen in einer gemeinsamen Transaktion. Eine Partie hat höchstens ein endgültiges Ergebnis. Eine vollständige Prüfung der Schachregeln ist nicht verlangt. Turniere, Einladungen oder Ranglisten eignen sich als Erweiterung.

Geeignete Hauptressource: `Game`.

10. Lieferando

Modelliert einen Lieferdienst. Restaurants pflegen Speisekarten mit zeitlich verfügbaren Gerichten. Eine Bestellung gehört zu einem Kunden, einer Lieferadresse und genau einem Restaurant. Bestellpositionen speichern Menge und Preis zum Bestellzeitpunkt. Der Status wechselt nur entlang eines festgelegten Ablaufs, zum Beispiel von `RECEIVED` über `PREPARING` und `DELIVERING` zu `DELIVERED`. Eine stornierte Bestellung darf nicht mehr ausgeliefert werden.

Geeignete Hauptressource: `Order`.

11. Spotify

Modelliert einen Musikdienst. Künstschaaffende veröfentlicen Titel, die zu einem Album gehören können. Konten erstellen Playlists und ordnen Titel darin an. Derselbe Titel darf mehrfach in einer Playlist vorkommen, deshalb braucht jede Position eine eigene Identität oder einen zusammengesetzten Schlüssel. Wiedergaben speichern Zeitpunkt und abgespielte Dauer. Gemeinsame Playlists haben mehrere Mitglieder mit Rollen. Die Musikdateien selbst und die Abrechnung von Lizenzen liegen außerhalb des Projekts.

Geeignete Hauptressource: `Playlist` oder `Track`.

Verbindliche Anforderungen

Die Bezeichnungen A0 bis C3 entsprechen den Übungen. Nutzt die vorhandenen Aufgaben als Arbeitsweg, übernehmt aber keine Musterlösung unverändert.

A0 bis A3: Fachfall und Datenmodell

1. Beschreibt Zweck, Nutzerkreis und Grenze eurer Anwendung.
2. Haltet mindestens fünf konkrete Geschäftsregeln fest. Mindestens zwei Regeln müssen später durch Constraints oder Anwendungslogik geprüft werden.
3. Erstellt ein ER-Modell mit mindestens vier Grundentitäten und einer zusätzlichen fachlichen Entität pro Gruppenmitglied. Plant insgesamt etwa fünf bis neun Entitäten. Das Modell enthält Primärschlüssel, Kardinalitäten, Optionalitäten, mindestens eine 1:n-Beziehung und mindestens eine n:m-Beziehung. Die n:m-Beziehung erhält im Relationenmodell eine eigene Zwischentabelle.
4. Überführt das ER-Modell in ein Relationenmodell. Markiert Primär-, Fremd- und fachliche eindeutige Schlüssel.
5. Untersucht eine breite Ausgangsrelation auf funktionale Abhängigkeiten. Zeigt eine konkrete Änderungsanomalie und begründet die Zerlegung bis zur dritten Normalform.
6. Diskutiert eine mögliche Denormalisierung für eine häufige Abfrage. Setzt sie nur um, wenn ihr den Nutzen mit einem Abfrageplan oder einer Messung belegen könnt.

B1 bis B4: Relationale Datenbank und Zugriff

1. Legt das Schema mit Flyway-Migrationen an. Verwendet Primär- und Fremdschlüssel sowie passende `NOT NULL`-, `UNIQUE`- und `CHECK`-Constraints.
2. Stellt reproduzierbare Seed-Daten bereit. Für die Hauptressource und jede von ihr direkt benötigte Tabelle sind mindestens drei fachlich unterschiedliche Datensätze nötig.
3. Legt eine Datei `queries.sql` mit vier lesbaren Abfragen an:
 - Filter und Sortierung auf der Haupttabelle
 - `INNER JOIN`
 - `LEFT JOIN`
 - Aggregation mit `GROUP BY`
4. Kapselt den Datenzugriff in einem Repository. Controller und Service dürfen keine JDBC-Typen kennen.
5. Verwendet parametrisierte Abfragen. SQL darf nicht durch das Aneinanderhängen ungeprüfter Request-Werte entstehen.
6. Legt mindestens zwei fachlich begründete Indizes zusätzlich zu Primär- und eindeutigen Schlüsseln an. Vergleicht für eine passende Abfrage den Plan vor und nach dem Index, zum Beispiel mit `EXPLAIN QUERY PLAN`.
7. Implementiert eine Business-Regel, die mehrere Lese- oder Schreibschritte umfasst. Wenn ihr dabei mehrere Tabellen ändert, führt die Änderungen in einer Transaktion aus.

C1 bis C3: REST-API

Implementiert die Anwendung mit Java 21, Spring Boot, Spring Web, Validation, Spring JDBC oder JDBC, SQLite und Flyway. JPA ist nicht Teil der Pflichtaufgabe.

Die Hauptressource braucht folgende Endpunkte:

Methode	Pfad	Verhalten
GET	/api/<ressourcen>	Liste lesen, 200 OK
GET	/api/<ressourcen>/{id}	Ein Element lesen, 200 OK oder 404 Not Found
POST	/api/<ressourcen>	Element anlegen, 201 Created mit Location

Jedes Gruppenmitglied übernimmt mindestens eine gekennzeichnete fachliche Erweiterung und die zugehörige CRUD-Operation in der REST-API. Die Erweiterung nutzt die zusätzliche Entität dieser Person. Beispiele sind PUT, DELETE, eine gefilterte Suche mit JOIN, ein Statuswechsel oder eine Unterressource. Mindestens eine Erweiterung der Gruppe enthält eine Abfrage mit JOIN oder Aggregation. Ordnet die Erweiterungen im Bericht namentlich zu. Alle Gruppenmitglieder müssen trotzdem die gesamte Anwendung erklären können.

Für die API gelten außerdem diese Anforderungen:

- Request- und Response-DTOs sind von Datenbank- und Domänenklassen getrennt.
- Bean Validation prüft syntaktisch ungültige Eingaben.
- Eine unbekannte ID liefert 404 Not Found.
- Eine fachlich gültige, aber kollidierende Eingabe liefert 409 Conflict. Legt den Konflikt für euer Thema konkret fest.
- Der Fehlerkörper enthält mindestens code, message und correlationId. Validierungsfehler nennen zusätzlich die betroffenen Felder.
- Technische Ausnahme- und SQL-Texte erscheinen nicht im HTTP-Response.
- Eine OpenAPI-Datei oder die generierte OpenAPI-Beschreibung dokumentiert Requests, Responses, Statuscodes und Beispiele.
- requests.http oder ein gleichwertiges Skript enthält nachvollziehbare Beispielaufufe.

Tests und Ausführbarkeit

Das Projekt muss sich auf einem frisch ausgecheckten Stand ohne manuelles Anlegen von Tabellen starten lassen. ./gradlew test führt alle Tests aus. ./gradlew bootRun startet die API.

Automatisiert mindestens diese Fälle:

1. Repository liest einen vorhandenen Datensatz korrekt.
2. GET auf eine vorhandene ID liefert 200 und den erwarteten Response.
3. GET auf eine unbekannte ID liefert 404.
4. Ein gültiger POST liefert 201 und einen Location-Header.
5. Ein ungültiger Request liefert 400 mit Feldfehlern.
6. Der festgelegte Konflikt liefert 409.
7. Jede personenbezogene Erweiterung wird in mindestens einem Test geprüft.

Ein Frontend, Anmeldung, Rollen, Docker und Deployment sind nicht erforderlich. Ihr könnt solche Teile ergänzen, sie ersetzen aber keine Pflichtanforderung.

Abgabe

Gebt ein Git-Repository oder ein Archiv mit folgender nachvollziehbarer Struktur ab. Gleichwertige Strukturen sind möglich, wenn die Dateien schnell auffindbar sind.

```
projekt/
├── README.md
├── docs/
│   ├── projektbericht.pdf
│   ├── er-modell.pdf oder er-modell.png
│   └── openapi.yaml
├── src/
├── build.gradle.kts
├── gradlew
├── gradle/
├── queries.sql
└── requests.http
```

Die Abgabe enthält keine Zugangsdaten, Build-Verzeichnisse oder lokalen Datenbankdateien. Das Projekt-README nennt den Start- und Testbefehl sowie die benötigte Java-Version.

Projektbericht

Der Bericht umfasst zwei bis vier Inhaltsseiten als PDF. Deckblatt, Quellenverzeichnis und ein kurzer Anhang zählen nicht zum Umfang. Druckt dort keine Quelltexte ab. Kleine Diagrammausschnitte, Tabellen und einzelne Requests sind sinnvoll, wenn ihr im Text darauf eingeht.

Beantwortet knapp und konkret:

1. Welches Problem löst die Anwendung, und was liegt außerhalb des gewählten Umfangs?
2. Welche Geschäftsregeln haben das ER- und Relationenmodell bestimmt?
3. Welche Anomalie beseitigt eure wichtigste Normalisierungsentscheidung, und wann wäre eine Denormalisierung vertretbar?
4. Wie läuft ein beispielhafter Request durch Controller, Service und Repository bis zur Datenbank?
5. Welchen Konflikt bildet die API mit 409 ab, und wie weist ein Test das Verhalten nach?
6. Wo braucht ihr eine Transaktion oder mehrschrittige Business-Regel?
7. Für welche Abfrage helfen eure Indizes? Belegt die Antwort mit dem Abfrageplan.
8. Welche personenbezogenen Erweiterungen habt ihr umgesetzt?
9. Warum passt eine relationale Datenbank zu eurem Fachfall? Nennt auch eine konkrete Teilaufgabe, für die ein anderes Datenmodell in Frage käme.
10. Was sollte ein nachfolgendes Team als Erstes verbessern oder ergänzen, und worauf stützt ihr diese Empfehlung?

Nennt verwendete Quellen, fremden Code und eingesetzte Hilfsmittel. Gruppen mit zwei oder drei Personen ergänzen eine kurze Tabelle, die die Arbeitsschwerpunkte der Mitglieder beschreibt. Alle Gruppenmitglieder bleiben für die gesamte Abgabe verantwortlich.

Präsentation und Fachgespräch

Für jede Gruppe stehen zehn Minuten Präsentationszeit und fünf Minuten für Fragen zur Verfügung. Alle Gruppenmitglieder übernehmen einen erkennbaren Teil.

Zeigt in der Präsentation:

- den Fachfall und eine Geschäftsregel, die euer Modell geprägt hat,
- den betreffenden Ausschnitt aus ER- oder Relationenmodell,
- einen erfolgreichen Request durch alle Schichten,
- einen Fehlerfall mit passendem Statuscode,
- das Ergebnis des Testlaufs,
- die personenbezogenen Erweiterungen und ihre Zuordnung.

Führt die Anwendung live vor. Haltet für technische Probleme vorbereitete Requests und aussagekräftige Screenshots bereit. Das Fachgespräch kann jede Stelle der gemeinsamen Lösung betreffen.

Bewertung mit den offiziellen Bögen

Für die Bewertung verwenden wir die beiden offiziellen Hochschulbögen für Projektbericht und Präsentation. Beide Bögen sind auf jeweils 30 Punkte ausgelegt. Für jedes Kriterium wird eine Tendenz von --, -, 0, + oder ++ markiert. Beim Projektbericht gibt es zusätzlich die Auswahl "nicht relevant". Die Punkte- und Notenberechnung erfolgt direkt im offiziellen Bogen. Es gibt deshalb keine davon abweichende 100-Punkte-Matrix in dieser Aufgabe.

Die Implementierung ist trotzdem ein verbindlicher Teil der Abgabe. Modell, Migrationen, API und Tests liefern die überprüfbaren Ergebnisse für den Bericht und die Live-Demo. Wenn ein beschriebener Endpunkt nicht läuft oder ein Modell nicht zum Schema passt, fehlt der Nachweis an mehreren Stellen des Bewertungsbogens.

Projektbericht, maximal 30 Punkte

Offizielles Kriterium	Woran es bei dieser Abgabe sichtbar wird
A. Gliederung, Struktur, Roter Faden	Der Bericht führt auf zwei bis vier Seiten nachvollziehbar vom Fachfall über die Entscheidungen zum geprüften Ergebnis.
B. Erläuterung Zielsetzung(en) des Projekts	Problem, Nutzerkreis, Projektgrenze, Hauptressource und Geschäftsregeln sind konkret beschrieben.
C. Erläuterung der Projektschritte	Der Bericht erklärt die Schritte vom ER-Modell über Schema und Repository bis zur REST-API. Eine reine Tätigkeitsliste reicht nicht.
D. Diskussion der eingesetzten Methoden	Ihr begründet Normalisierung, Constraints, Transaktion, Indizes, Schichtentrennung und Tests anhand eures Fachfalls.
E. Erläuterung wesentlicher Ergebnisse des Projekts	Modell, laufende Endpunkte, Fehlerfälle, Abfrageplan und Testergebnis werden mit kleinen, lesbaren Belegen erklärt.
F. Darlegung abgeleiteter Handlungsempfehlungen	Ihr benennt eine sinnvolle nächste Änderung und leitet sie aus einer Grenze, einem Testergebnis oder einer bewussten Vereinfachung ab.

Offizielles Kriterium	Woran es bei dieser Abgabe sichtbar wird
G. Darlegung Einsatz von Methoden und Werkzeugen	Ihr nennt den konkreten Einsatz von Java, Spring Boot, JDBC, SQLite, Flyway, OpenAPI und euren Testwerkzeugen.
H. Berücksichtigung wissenschaftlicher Erkenntnisse	Aussagen zu Normalformen, relationalem Entwurf und HTTP-Verhalten sind fachlich richtig und mit geeigneten Quellen belegt.
J. Vollständigkeit, Form und Layout	Umfang, Lesbarkeit, Quellen, Abbildungen, Arbeitsschwerpunkte und alle geforderten Abgabeteile stimmen.

Ein schöner Bericht kann eine fehlende Implementierung nicht verdecken. Umgekehrt erklärt sich guter Code nicht von selbst. Der Bericht muss die wichtigen Entscheidungen und Ergebnisse so zeigen, dass sie auf dem offiziellen Bogen beurteilt werden können.

Präsentation, maximal 30 Punkte

Offizielles Kriterium	Woran es bei dieser Abgabe sichtbar wird
A. Gliederung, Struktur, Roter Faden	Die Präsentation folgt einem klaren Weg vom Fachfall über eine Modellentscheidung zur Live-Demo und bleibt innerhalb der Zeit.
B. Fachwissen, Kritische Reflexion	Ihr erklärt, warum Modell, Constraints, Statuscodes und Zusatzfunktionen so umgesetzt sind, und spricht über echte Grenzen.
C. Visualisierung	ER-Ausschnitt, Request und Response, Abfrageplan sowie Testergebnis sind lesbar. Die Live-Demo zeigt den behaupteten Ablauf.
D. Auftreten, Blickkontakt, Sprache	Ihr sprecht verständlich, lest nicht nur Folien ab und teilt die Präsentation in der Gruppe sinnvoll auf.
E. Diskussion	Im Fachgespräch erklärt jedes Gruppenmitglied Entscheidungen und Code der gemeinsamen Lösung, nicht nur den eigenen Arbeitsanteil.

Die Live-Demo wirkt vor allem auf Fachwissen, Visualisierung und Diskussion. Ein vorbereiteter Screenshot hilft bei einem technischen Ausfall, ersetzt aber keine ausführbare Abgabe.

Abgabecheck

- | | |
|--|---|
| <ul style="list-style-type: none"> ☐ Gruppe mit ein bis drei Personen und Thema festgelegt ☐ Bericht mit zwei bis vier Inhaltsseiten als PDF ☐ ER-Modell, Relationenmodell und Normalisierungsbegründung vorhanden ☐ Flyway baut Schema und Seed-Daten selbstständig auf ☐ <code>queries.sql</code> enthält vier geforderte Abfragen ☐ eine zusätzliche Entität und eine gekennzeichnete REST-Operation pro Person umgesetzt ☐ mindestens eine komplexe Abfrage in der API verwendet | <ul style="list-style-type: none"> ☐ zwei Indizes mit Abfrageplan begründet ☐ Business-Regel und gegebenenfalls Transaktion umgesetzt ☐ <code>400</code>, <code>404</code> und <code>409</code> nachvollziehbar umgesetzt ☐ sechs Basisfälle und jede personenbezogene Erweiterung automatisiert getestet ☐ <code>./gradlew test</code> läuft auf einem frischen Stand ☐ OpenAPI-Beschreibung und Beispielrequests vorhanden ☐ Präsentation und Ersatzmaterial für die Demo vorbereitet ☐ Quellen, fremder Code und Hilfsmittel angegeben |
|--|---|